AI-Powered QA Testing Agent — Progress Report

Date: March 18, 2026 **Application Under Test:** TECU Credit Union — New Member Application **URL:** qa-tq-awp.impactodigifin.xyz/newapplication **Page Tested:** Contact Info (Page 1 of 6)

Executive Summary

We have built an AI agent that autonomously tests web applications. Given only a URL, it opens a browser, explores the page, fills fields, learns the page structure, and then runs validation test cases to find bugs — all without any pre-written scripts or manual test steps.

> **Key Result:** The agent autonomously designed **30 test cases**, executed **13**, and discovered **3 high-severity validation defects** — all within **5 minutes** at a cost of **~₹28 ($0.30)** per run.

How It Works

The agent operates in two passes:

Phase	What the Agent Does	Time	Cost
Pass 1: Explore & Learn (One-time)	Opens the page, discovers all fields, fills them with valid data, extracts element identifiers, saves a knowledge file	~3 min	₹26 (One-time)
Pass 2a: Plan Test Cases	Reads the knowledge file, autonomously designs 30 test cases with priorities	~10 sec	₹5
Pass 2b: Execute Tests	Runs each test case: enters invalid data, checks for errors, records pass/fail	~4 min	₹23

The agent decides what to test based on what it learned.

Performance Metrics

Two test runs were conducted back-to-back on the same page to verify consistency:

Metric	Run 1	Run 2	Average
Test cases planned	31	30	**30**
Tests executed	13	13	**13**
Passed	5	5	**5**
Failed (bugs found)	7	8	**8**
Skipped (time limit)	18	17	**17**

Metric	Run 1	Run 2	Average
Cost	₹27.95	₹27.74	**₹27.85**
Duration	248s	303s	**275s (~4.6 min)**
Cache efficiency	54.2%	49.3%	**51.8%**

Test Case Results (Run 2 — Latest)

Executed Tests

#	Field	Test Case	Input	Expected	Actual	Status
1	First Name*	Empty required field	""	Error shown	NO_ERROR	FAIL
2	Last Name*	Empty required field	""	Error shown	NO_ERROR	FAIL
3	Email ID*	Empty required field	""	Error shown	NO_ERROR	FAIL
4	Email ID*	Invalid email format	"notanemail"	Error shown	NO_ERROR	FAIL
5	First Name*	Numbers in name field	"12345"	Error shown	NO_ERROR	FAIL
6	First Name*	Special characters	"!@#%"	Error shown	NO_ERROR	FAIL
7	Last Name*	Numbers in name field	"12345"	Error shown	NO_ERROR	FAIL
8	Last Name*	Special characters	"!@#%"	Error shown	NO_ERROR	FAIL
9	First Name*	Valid name accepted	"Roman"	No error	NO_ERROR	PASS
10	First Name*	Field stays editable	"Roman"	No error	NO_ERROR	PASS
11	Last Name*	Valid name accepted	"Tester"	No error	NO_ERROR	PASS
12	Last Name*	Field stays editable	"Tester"	No error	NO_ERROR	PASS
13	Email ID*	Plus-addressing works	"roman.qa+tecu@example.com"	No error	NO_ERROR	PASS

Result Distribution

```
PASSED   [=====              ] 5/13  (38%)  - App behaved correctly
FAILED   [==========         ] 8/13  (62%)  - Bugs found
SKIPPED  [=================]  17    - Not reached (turn limit)
```

Consistency Across Runs

Both runs found the **same core defects** with identical behavior, confirming the results are reliable and reproducible, not random.

Bugs Discovered

The agent found that the application has **no client-side validation** on the Contact Info page. All required fields accept empty values, and name fields accept numbers and special characters without any error message.

#	Field	Issue Found	Severity	What the Agent Did
1	**First Name**	Accepts empty value, numbers ("12345"), and symbols ("!@#%") without error	**HIGH**	Set field to empty, numeric, and special char values — no error appeared after any
2	**Last Name**	Accepts empty value, numbers ("12345"), and symbols ("!@#%") without error	**HIGH**	Same tests as First Name — identical lack of validation
3	**Email ID**	Accepts empty value and "notanemail" (no @ symbol) without error	**HIGH**	Set field to empty and invalid format — no email format validation exists

> **Note:** These findings are for client-side (on-blur) validation only. The agent was instructed not to click "Save & Continue", so server-side validation was not tested. If the application validates on submit only, these may be by design, but best practice is to provide immediate feedback to users.

Cost Breakdown

Pass 1 (one-time per page): Runs once to learn the page. Reused by all subsequent test runs.

Component	Cost (₹)	Cost ($)	Frequency
Pass 1: Explore & Learn	₹26.23	$0.29	**One-time per page**

Pass 2 (per test run): Each test run uses the saved knowledge from Pass 1.

Component	Cost (₹)	Cost ($)	% of Run Cost

Component	Cost (₹)	Cost ($)	% of Run Cost
Pass 2a: Plan Test Cases	₹5.00	$0.06	18%
Pass 2b: Execute Tests	₹18.00	$0.20	64%
Pass 2b: Generate Report	₹5.00	$0.05	18%
Total per test run	**₹28.00**	**$0.30**	**100%**

Cost Metric	Value
Cost per test case executed	₹2.14 ($0.023)
Cost per bug discovered	₹9.33 ($0.10)
First run cost (Pass 1 + Pass 2)	₹54 ($0.59)
Every subsequent run (Pass 2 only)	₹28 ($0.30)
Cache savings per run	~28% (₹10-12 saved)

What Works Today vs What's Next

Working Now	In Progress
Text field testing (fill, validate, edge cases)	Mobile Number selector fix
Email format validation testing	Branch/dropdown testing via JS
Autonomous test case generation (30 cases)	Turn limit increase (execute all 30 tests)
XPath extraction for all elements	HTML visual reports
Bug detection with evidence	
Works on any URL (tested: DemoQA, ParaBank, TECU)	

Next Steps

#	Action	Impact
1	Fix Mobile Number & Branch selectors	100% field coverage on Page 1
2	Increase turn limit to 40	Execute all 30 planned test cases
3	Auto-generate HTML visual reports	Stakeholder-ready output per run
4	Run full 6-page TECU application test	Complete application coverage
5	Reduce output token cost	Target: ₹35/page (35% reduction)
6	SQLite cross-run memory	Agent learns from past failures

Technology Stack

Component	Technology	Role
AI Model	GPT-5 (OpenAI)	Understands pages, plans tests, makes decisions
Agent Framework	OpenAI Agents SDK	Manages the AI's tool calls and conversation loop
Browser Control	Chrome DevTools MCP (Google)	Gives the AI direct access to Chrome — click, fill, inspect
Browser	Google Chrome	The actual browser the agent controls
Language	Python 3.10+	Orchestration, cost tracking, report generation
Cost Optimization	Custom compaction engine	Keeps AI memory small to reduce token costs

Report generated from AI QA Testing Agent v2 | Model: GPT-5 | SDK: OpenAI Agents | Browser: Chrome DevTools MCP